

Projekt 1

Simon Buschmann, Yannik Buchner - LLLF(MLU)

Nikita Podibko, Jan Draeger - ALU

Johannes Heinrich, Malek Haoues Rhaïem - APL

TU Dortmund

Inhaltsverzeichnis

1 Least Loaded Link First (MLU)

2 Random Load Aware

3 Average Path Length

Projekt 1

Simon
Buschmann,
Yannik Buchner -
LLLF(MLU)
Nikita Podibko,
Jan Draeger - ALU
Johannes Heinrich,
Malek Haoues
Rhaïem - APL

Least Loaded Link
First (MLU)

Random Load
Aware

Average Path
Length

Simon
Buschmann,
Yannik Buchner -
LLLF(MLU)
Nikita Podibko,
Jan Draeger - ALU
Johannes Heinrich,
Malek Haoues
Rhaïem - APL

Least Loaded Link First (MLU)

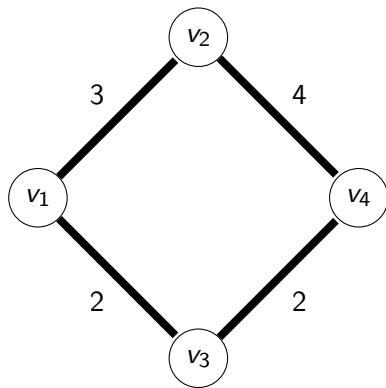
Least Loaded Link
First (MLU)

Random Load
Aware

Average Path
Length

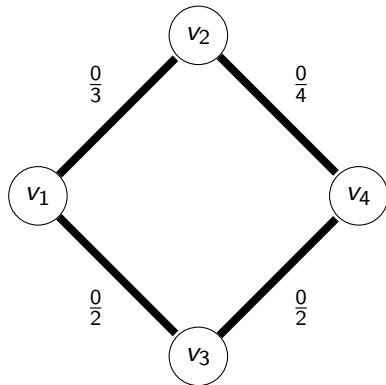
- Betrachte jeden Auftrag nacheinander einzeln.
- Nutze Dijkstra um für diesen Auftrag einen Pfad von s nach t zu bestimmen, der die Utilisation minimal hält.
- Nutze dabei nicht nur die capacity, sondern auch den aktuellen flow der vorangegangenen Aufträge.
- Behandle dabei nicht Knoten mit minimaler Distanz, sondern mit minimale Utilisation auf Pfad.

Beispiel



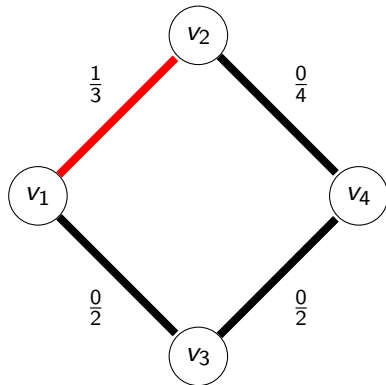
Beispiel

Auftrag 1: $s = 1$, $t = 4$, $demand = 1$



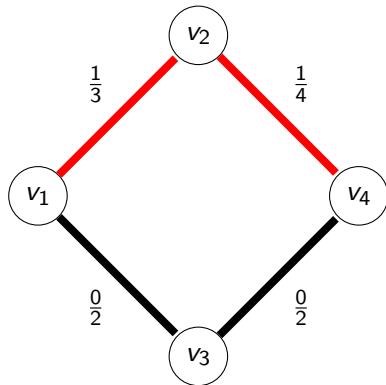
Beispiel

Auftrag 1: $s = 1$, $t = 4$, $demand = 1$



Beispiel

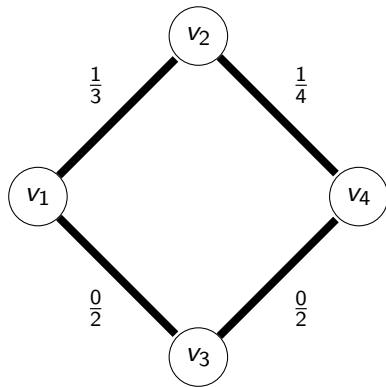
Auftrag 1: $s = 1$, $t = 4$, $demand = 1$



Beispiel - Fortsetzung

Auftrag 1: $s = 1$, $t = 4$, $demand = 1$

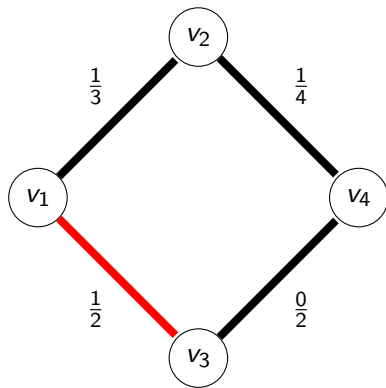
Auftrag 2: $s = 1$, $t = 4$, $demand = 1$



Beispiel - Fortsetzung

Auftrag 1: $s = 1$, $t = 4$, $demand = 1$

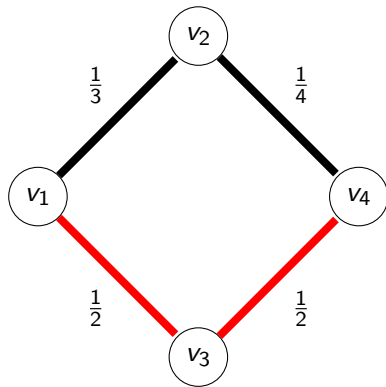
Auftrag 2: $s = 1$, $t = 4$, $demand = 1$



Beispiel - Fortsetzung

Auftrag 1: $s = 1$, $t = 4$, $demand = 1$

Auftrag 2: $s = 1$, $t = 4$, $demand = 1$



Algorithm 1 LLLFSolver

Require: $G(V, E, c : E \mapsto \mathbb{R}), D = \{D_1 = (s_1, t_1, d_1), D_2, \dots, D_n\}$

Ensure: $P = \{P_1, P_2, \dots, P_n\}$

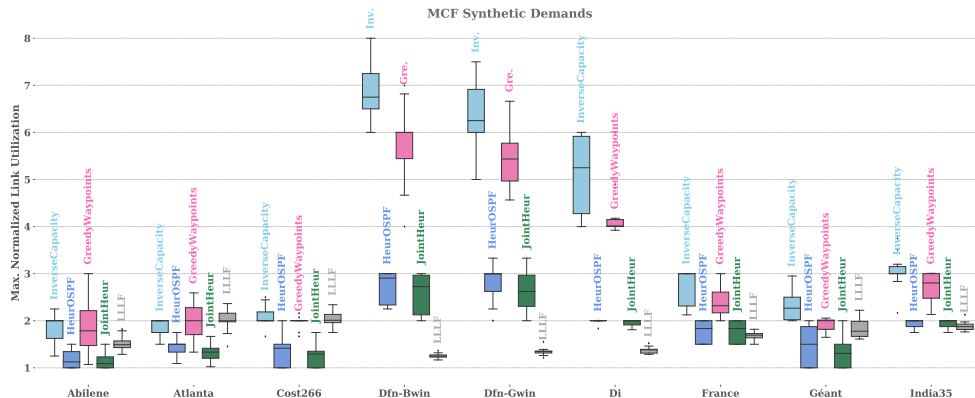
```
1: for  $e \in E$  do
2:    $loads_e \leftarrow 0$ 
3: end for
4:  $P \leftarrow \{\}$ 
5: for  $i \in \{1, \dots, n\}$  do
6:    $P_i \leftarrow PotentialUtilizationDijkstra(G, loads, D_i)$ 
7:   for  $e \in P_i$  do
8:      $loads_e \leftarrow loads_e + d_i$ 
9:      $utilization_e \leftarrow loads_e / c(e)$ 
10:  end for
11:   $P \leftarrow P \cup \{P_i\}$ 
12: end for
13: return  $P$ 
```

Algorithm 2 PotentialUtilizationDijkstra

Require: $G(V, E, c : E \mapsto \mathbb{R}), loads, D_i = (s_i, t_i, d_i)$

Ensure: P_1

```
1: for  $v \in V$  do
2:    $utilization_v \leftarrow \infty$ 
3:    $predecessor_v \leftarrow -1$ 
4:    $visited_v \leftarrow \text{False}$ 
5: end for
6:  $utilization_{s_i} \leftarrow 0$ 
7: while  $visited_{t_i} == \text{False}$  do
8:    $u \leftarrow v$  with minimal  $utilization_v$ 
9:   for  $n \in neighbours(u)$  do
10:    if  $utilization_n > \max\{utilization_u, (loads_{(u,n)} + d_i)/c(u, n)\}$  then
11:       $predecessor_n \leftarrow u$ 
12:       $utilization_n \leftarrow \max\{utilization_u, (loads_{(u,n)} + d_i)/c(u, n)\}$ 
13:    end if
14:  end for
15:   $visited_u \leftarrow \text{True}$ 
16: end while
17:  $P_1 \leftarrow$  Generate Path with  $predecessor$  starting at  $predecessor_t$ 
18: return  $P_1$ 
```

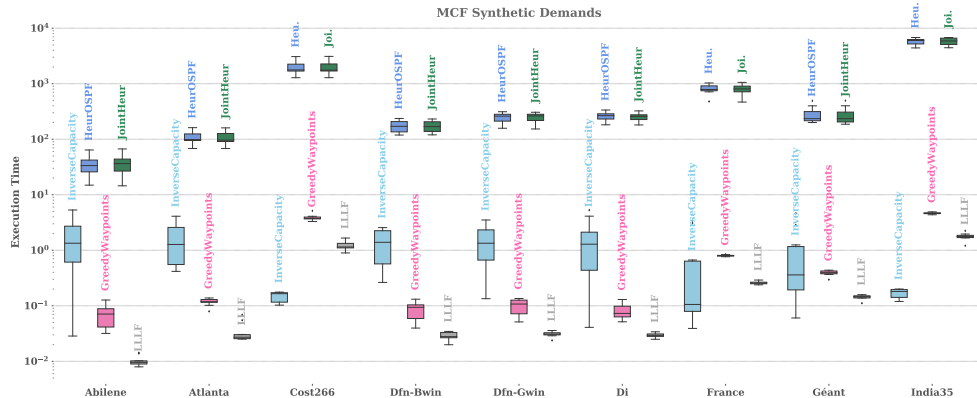


Least Loaded Link
First (MLU)

Random Load
Aware

Average Path
Length

Ergebnisse



Projekt 1

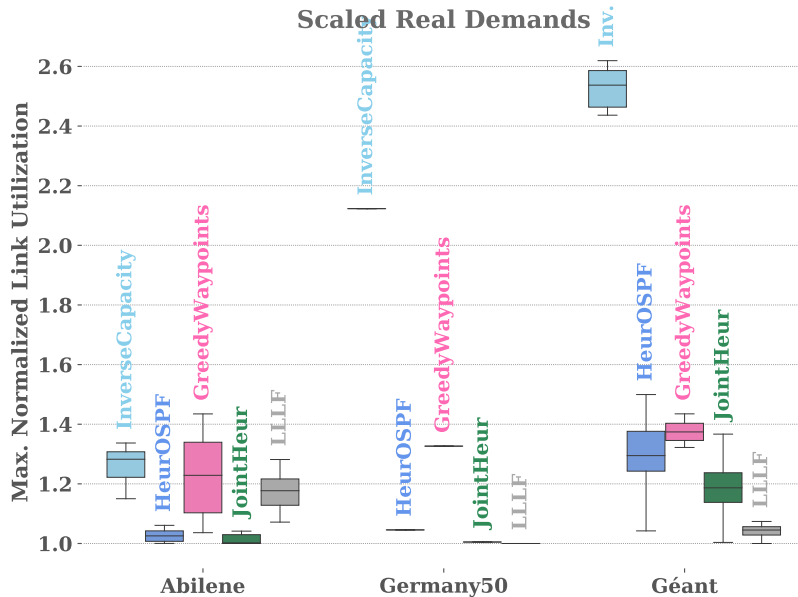
Simon
Buschmann,
Yannik Buchner -
LLLF(MLU)
Nikita Podibko,
Jan Draeger - ALU
Johannes Heinrich,
Malek Haoques
Rhaïem - APL

Least Loaded Link
First (MLU)

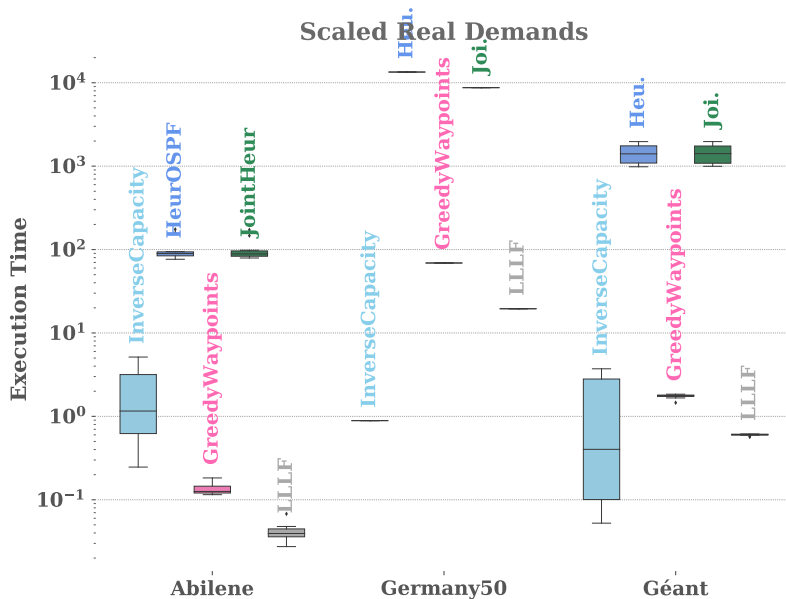
Random Load
Aware

Average Path
Length

Ergebnisse



Ergebnisse



Projekt 1

Simon
Buschmann,
Yannik Buchner -
LLLF(MLU)
Nikita Podibko,
Jan Draeger - ALU
Johannes Heinrich,
Malek Haoes
Rhaïem - APL

Least Loaded Link
First (MLU)

Random Load
Aware

Average Path
Length

Vorteile:

- gut bei vielen kleinen demands
- gut bei relativ dichten Netzwerken
- schnell

Vorteile:

- gut bei vielen kleinen demands
- gut bei relativ dichten Netzwerken
- schnell

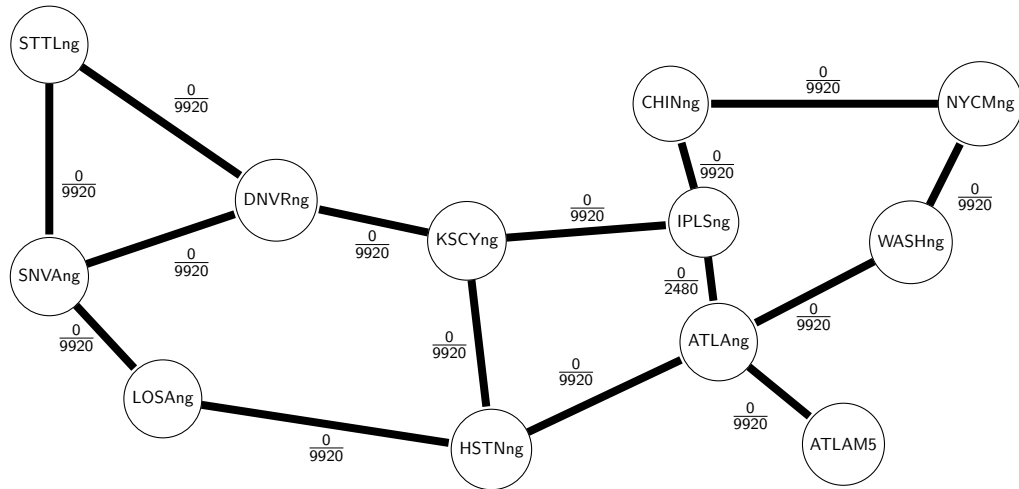
Nachteile:

- schlecht bei wenigen großen demands
- schlecht bei relativ dünne Netzwerken

Beispiel wenige große Demands

Auftrag 1: $s = \text{HSTNng}$, $t = \text{SNVAng}$, $\text{demand} = 9920$

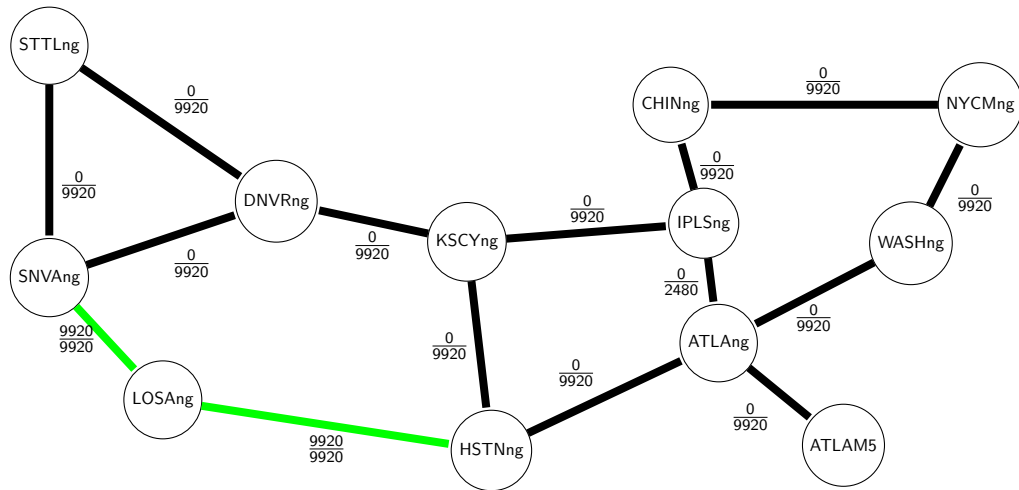
Auftrag 2: $s = \text{LOSang}$, $t = \text{SNVAng}$, $\text{demand} = 9920$



Beispiel wenige große Demands

Auftrag 1: $s = \text{HSTNng}$, $t = \text{SNVAng}$, $\text{demand} = 9920$

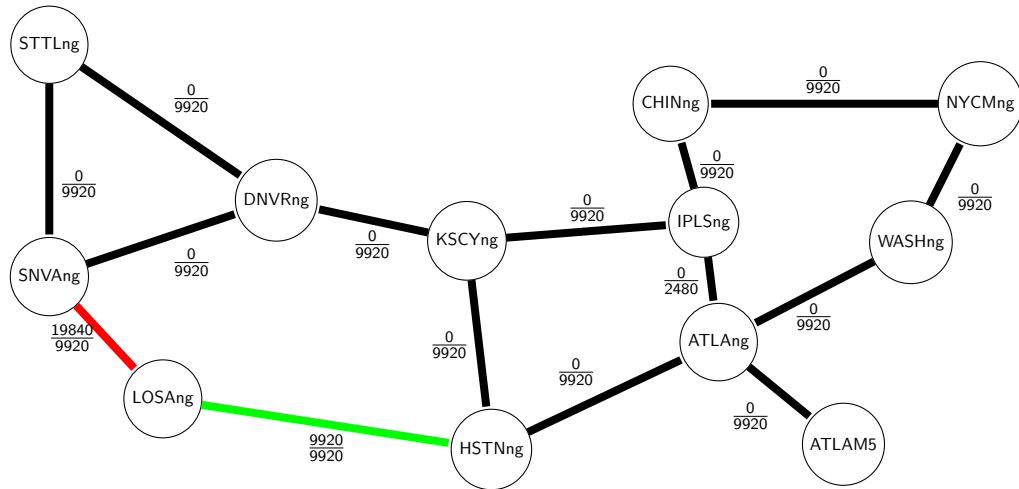
Auftrag 2: $s = \text{LOSang}$, $t = \text{SNVAng}$, $\text{demand} = 9920$



Beispiel wenige große Demands

Auftrag 1: $s = \text{HSTNng}$, $t = \text{SNVAng}$, $\text{demand} = 9920$

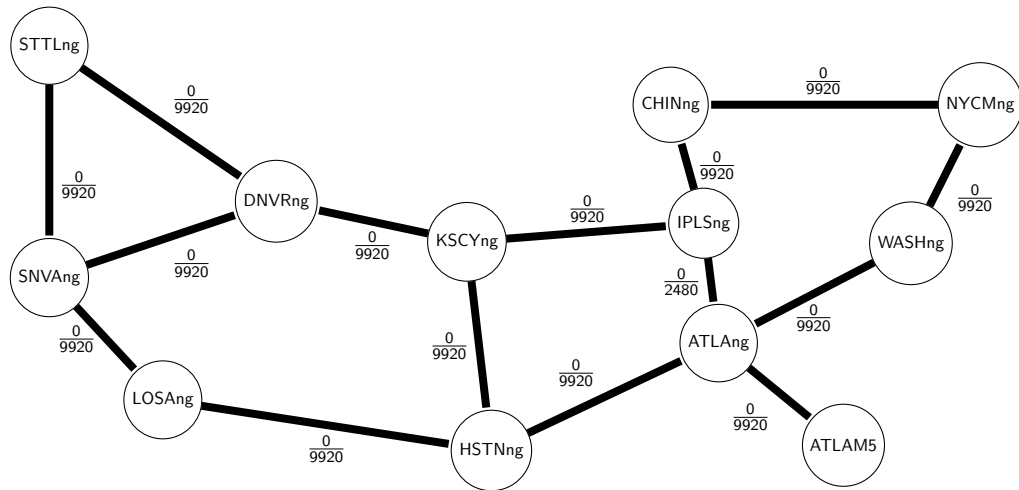
Auftrag 2: $s = \text{LOSang}$, $t = \text{SNVAng}$, $\text{demand} = 9920$



Beispiel viele kleinen Demands

Auftrag 1...10: $s = \text{HSTNng}$, $t = \text{SNVAng}$, $\text{demand} = 992$

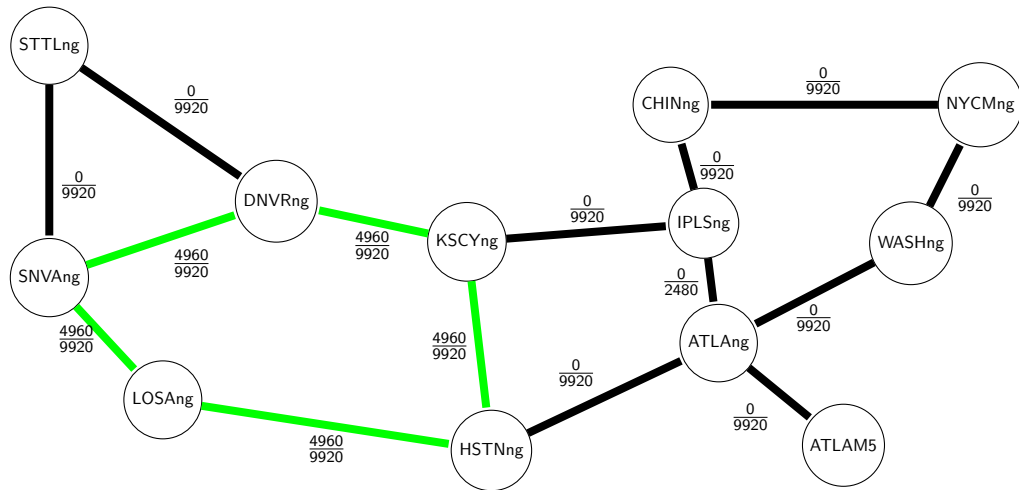
Auftrag 11...20: $s = \text{LOSang}$, $t = \text{SNVAng}$, $\text{demand} = 992$



Beispiel viele kleinen Demands

Auftrag 1...10: $s = \text{HSTNng}$, $t = \text{SNVAng}$, $\text{demand} = 992$

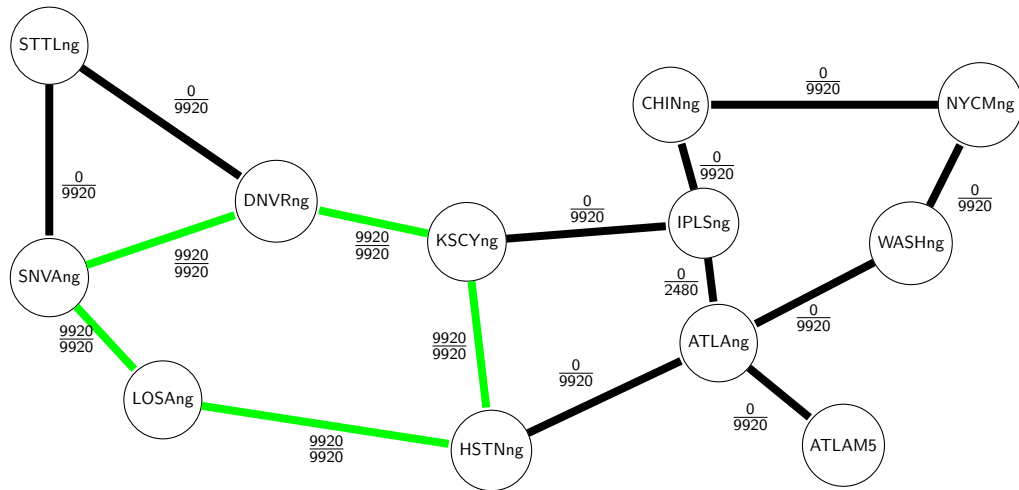
Auftrag 11...20: $s = \text{LOSang}$, $t = \text{SNVAng}$, $\text{demand} = 992$



Beispiel viele kleinen Demands

Auftrag 1...10: $s = \text{HSTNng}$, $t = \text{SNVAng}$, $\text{demand} = 992$

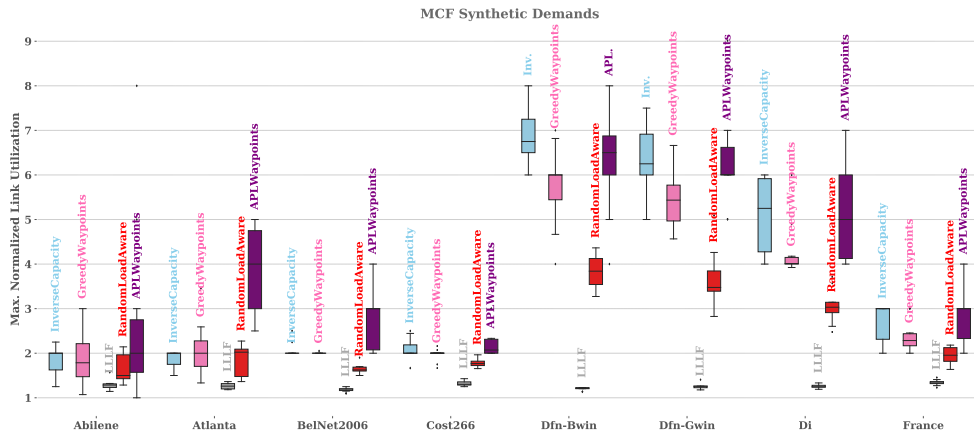
Auftrag 11...20: $s = \text{LOSang}$, $t = \text{SNVAng}$, $\text{demand} = 992$



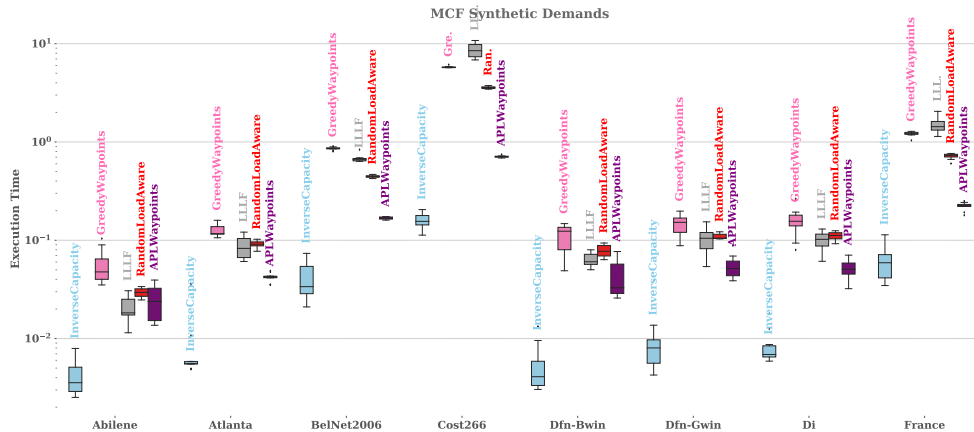
Weitere Verbesserungen:

- Liste der Demands absteigend sortieren
- Bei mehreren besten Pfaden den kürzesten nehmen
- Algorithmus mehrmals ausführen bis keine Verbesserung mehr entsteht (max. 10 Iterationen)

Ergebnisse



Ergebnisse



Projekt 1

Simon
Buschmann,
Yannik Buchner -
LLL(MLU)
Nikita Podibko,
Jan Draeger - ALU
Johannes Heinrich,
Malek Haoues
Rhaïem - APL

Least Loaded Link
First (MLU)

Random Load
Aware

Average Path
Length

Fazit:

- Recht guter Algorithmus für MLU
- Recht gute Laufzeit

Simon
Buschmann,
Yannik Buchner -
LLLF(MLU)
Nikita Podibko,
Jan Draeger - ALU
Johannes Heinrich,
Malek Haoues
Rhaïem - APL

Least Loaded Link
First (MLU)

Random Load
Aware

Average Path
Length

Random Load Aware

- Ziel: eine Methode, die mit überschaubarem Aufwand implementierbar ist und dabei praxisnahe Ergebnisse liefert.
- heuristische Pfadwahl, die Linkauslastung berücksichtigt und einfache Strukturen nutzt
- Der Algorithmus soll eine realitätsnahe Simulation von Lastverteilung ohne komplexe Optimierungstools ermöglichen.

Algorithm 3 Randomized Load-Aware Path Selection

Require: Graph $G = (V, E)$, source node s , destination node d , number of samples K

Ensure: Least-loaded path from s to d

```
1: Initialize  $bestScore \leftarrow \infty$ 
2: Initialize  $bestPath \leftarrow \emptyset$ 
3: Generate  $K$  random simple paths (we choose from 30 shortest for time saving) from  $s$  to  $d$ 
4: for  $p \in P$  do
5:    $score \leftarrow 0$ 
6:   for edge  $e$  in path  $p$  do
7:      $score \leftarrow score + Utilization(e)$ 
8:   end for
9:   if  $score < bestScore$  then
10:     $bestScore \leftarrow score$ 
11:     $bestPath \leftarrow p$ 
12:   end if
13: end for
14: return  $bestPath$ 
```


Ergebnisse: Maximale Linkauslastung (MLU)

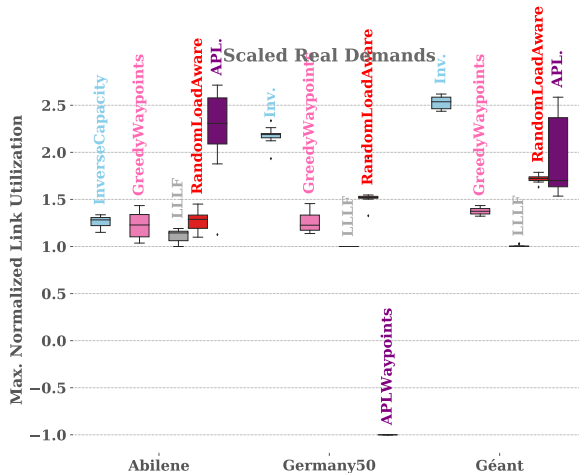


Abbildung: Maximale Normalisierte Linkauslastung pro Topologie

Ergebnisse: Durchschnittliche Linkauslastung (ALU)

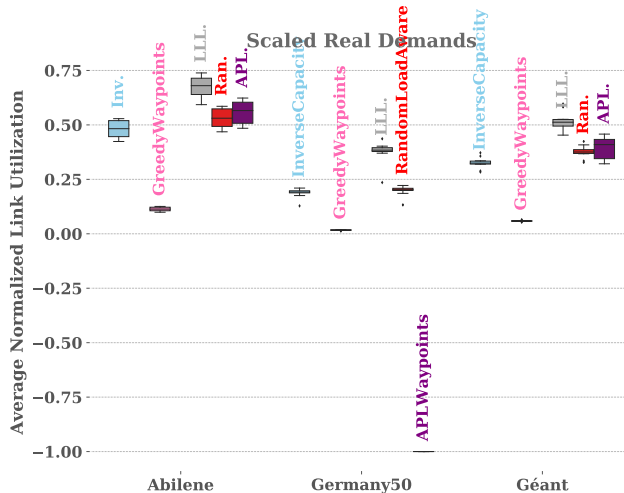


Abbildung: Durchschnittliche Normalisierte Linkauslastung (über alle Links)

Ergebnisse: Ausführungszeit

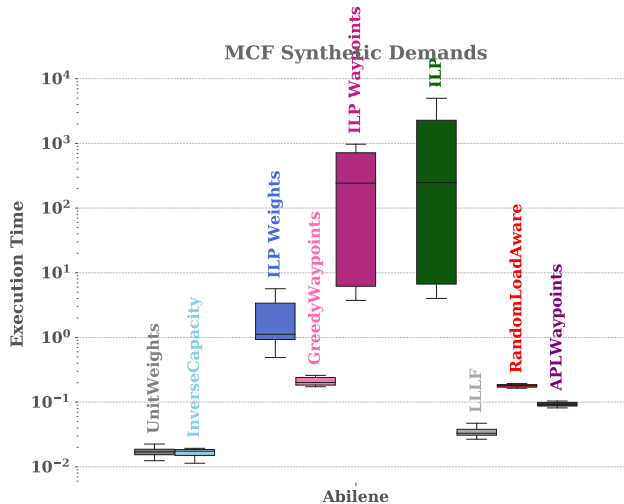


Abbildung: Vergleich der Laufzeiten verschiedener Algorithmen bei Real Demands

Ergebnisse: Maximale Linkauslastung (MLU)

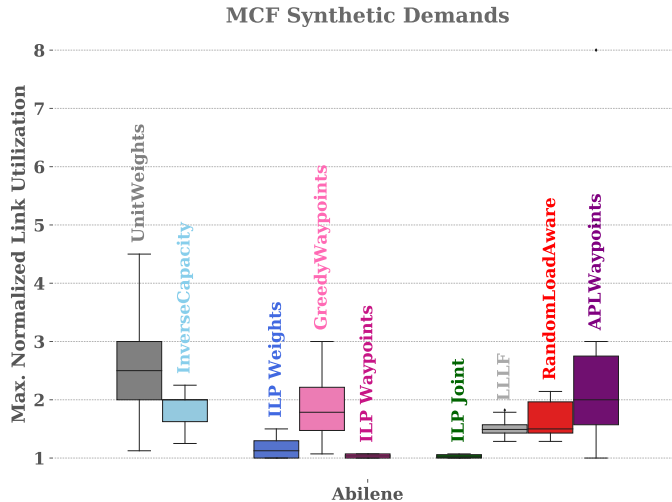


Abbildung: Maximale Normalisierte Linkauslastung pro Topologie

Ergebnisse: Durchschnittliche Linkauslastung (ALU)

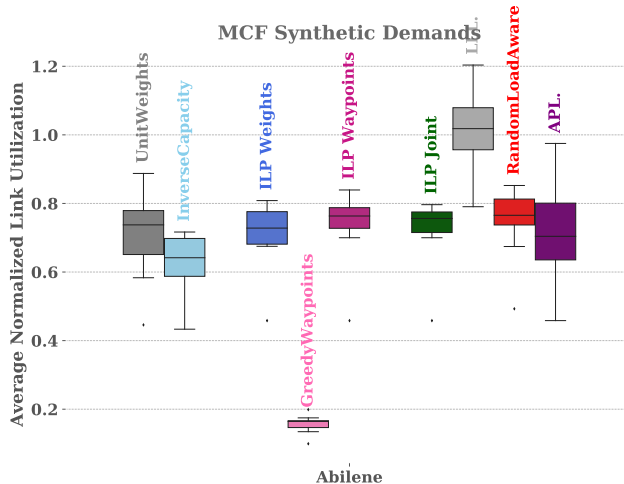
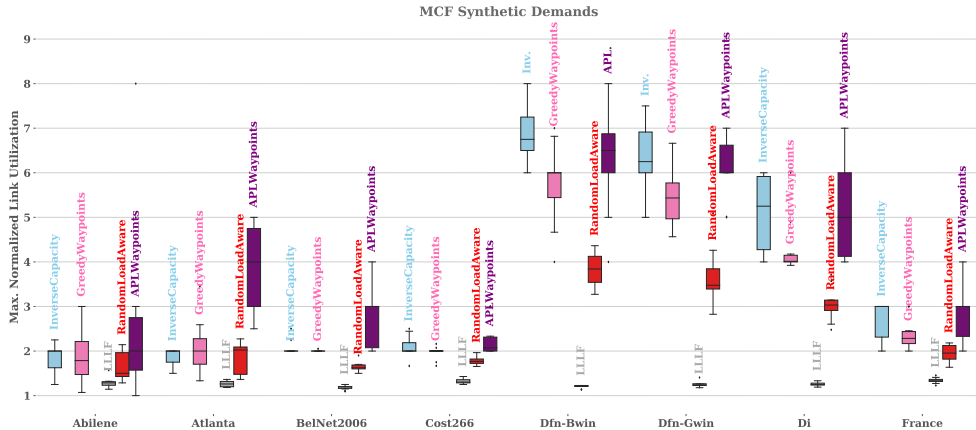


Abbildung: Durchschnittliche Normalisierte Linkauslastung (über alle Links)

Ergebnisse: Maximale Linkauslastung (MLU)

Projekt 1

Simon
Buschmann,
Yannik Buchner -
LLLF(MLU)
Nikita Podibko,
Jan Draeger - ALU
Johannes Heinrich,
Malek Haoues
Rhaïem - APL



Least Loaded Link
First (MLU)

Random Load
Aware

Average Path
Length

Abbildung: Maximale Normalisierte Linkauslastung alle Topologien

- **Lastbewusst:** berücksichtigt aktuelle Auslastung der Links
- **Effizienter als exakte Methoden:** Kein ILP, kein externer Solver notwendig
- **Einfach implementiert und modular:** gut verständlich und anpassbar
- **K-kompatibel:** nutzt mehrere (z. B. 3) Pfade pro Demand

- **Keine globale Optimierung:** Greedy-Ansatz, ignoriert zukünftige Demands
- **Nicht optimal:** kann in lokale Minima geraten
- **Keine Rücknahme:** einmal gesetzte Pfade bleiben fix
- **Begrenzter Suchraum:** prüft nur K Pfade – bessere bleiben evtl. unberücksichtigt

1. Pfadanzahl K adaptiv anpassen

- **Idee:** Passe die Anzahl der betrachteten Pfade dynamisch an – z. B. mehr Pfade bei hoher Netzlast.
- **Vorteil:** Höhere Lösungsqualität durch flexiblere Pfadwahl.
- **Nachteil:** Erhöhte Rechenzeit bei größeren K -Werten.

2. Lastbasierte Gewichtsaktualisierung

- **Idee:** Erhöhe temporär die Gewichtung ausgelasteter Links, um sie bei der Pfadwahl zu vermeiden.
- **Vorteil:** Verringert Hotspots, verbessert Lastverteilung.
- **Nachteil:** Zusätzlicher Rechenaufwand durch wiederholte Gewichtsaktualisierung.

Simon
Buschmann,
Yannik Buchner -
LLLF(MLU)
Nikita Podibko,
Jan Draeger - ALU
Johannes Heinrich,
Malek Haoues
Rhaïem - APL

Least Loaded Link
First (MLU)

Random Load
Aware

Average Path
Length

Average Path Length

- Optimierte die Pfadlängen für möglichst kurze Wege
- Gewichte die APL neben der MLU mittels Linearkombination
- Ziel: Optimalen Kompromiss zwischen kurzen Wegen und geringer Auslast finden

Algorithm 4 Berechne Average Path Length (APL)

Require: $D = \{(S_i, T_i, d_i)\}$, Pfadlängenfunktion $\text{len}(P_i)$

Ensure: APL-Wert

```
1: totalWeightedPathLength  $\leftarrow 0$ 
2: totalDemandWeight  $\leftarrow 0$ 
3: for jedes  $(s_i, t_i, d_i) \in D$  do
4:   if  $s_i \neq t_i$  and  $P_i$  exists then
5:      $P_i \leftarrow \text{berechnePfad}(s_i, t_i)$ 
6:      $\text{length} \leftarrow \text{len}(P_i)$ 
7:      $\text{totalWeightedPathLength} \leftarrow \text{totalWeightedPathLength} + \text{length} \cdot d_i$ 
8:      $\text{totalDemandWeight} \leftarrow \text{totalDemandWeight} + d_i$ 
9:   else
10:    continue
11:  end if
12: end for
13:  $\text{APL} \leftarrow \text{totalWeightedPathLength} / \text{totalDemandWeight}$ 
14: return APL
```

Formel für Linearkombination

$$\lambda \cdot w_{apl} + (1 - \lambda) \cdot mlu$$

- Je nach λ -Wert wird APL und MLU unterschiedlich gewichtet:
 - ① $\lambda = 0,5 \rightarrow$ gleiche Gewichtung von MLU und APL
 - ② $\lambda > 0,5 \rightarrow$ APL stärker gewichtet als MLU
 - ③ $\lambda < 0,5 \rightarrow$ MLU stärker gewichtet als APL

Ergebnisse: Laufzeit

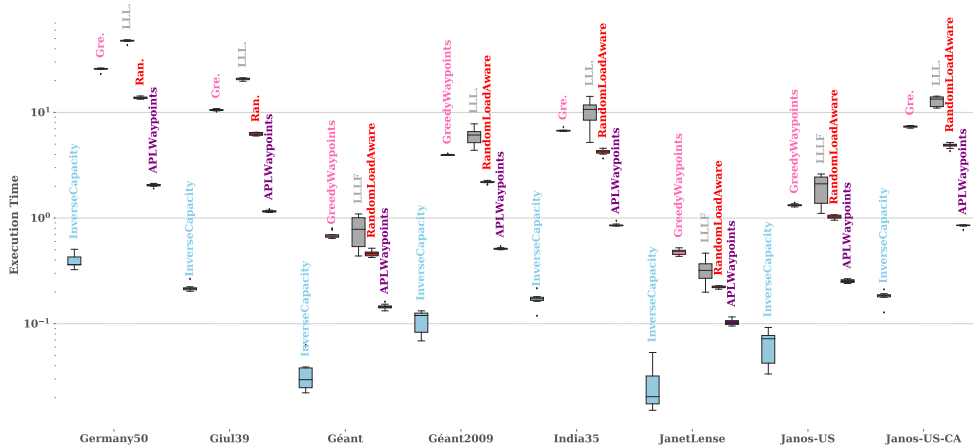


Abbildung: Execution time for each topology(MCF Synthetic Demands)

Ergebnisse: APL

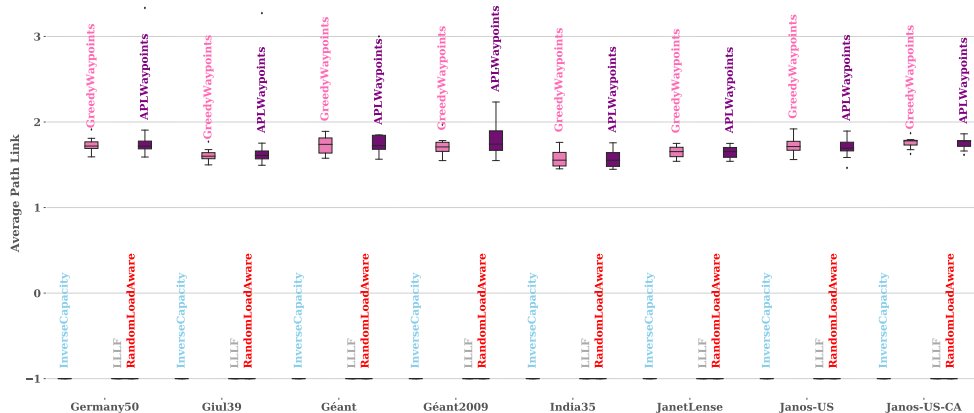


Abbildung: APL-Value for each topologie(MCF Synthetic Demands)

Ergebnisse: ALU

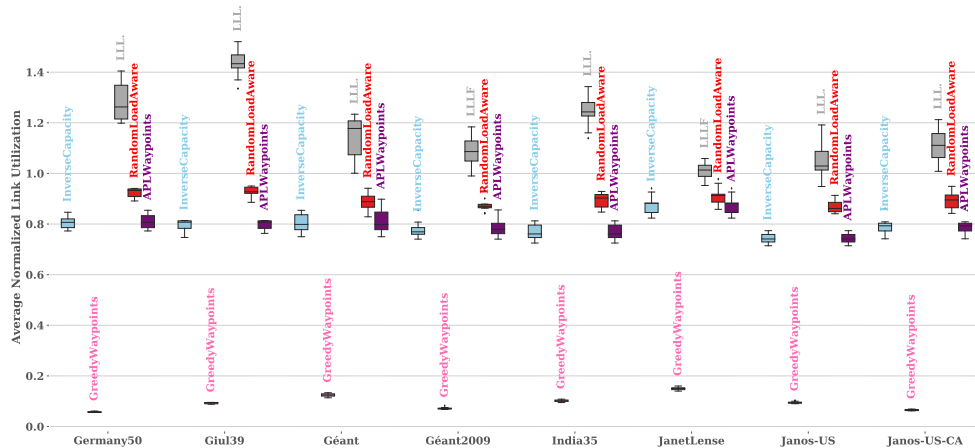


Abbildung: ALU-Value for each topologie(MCF Synthetic Demands)

Ergebnisse: MLU

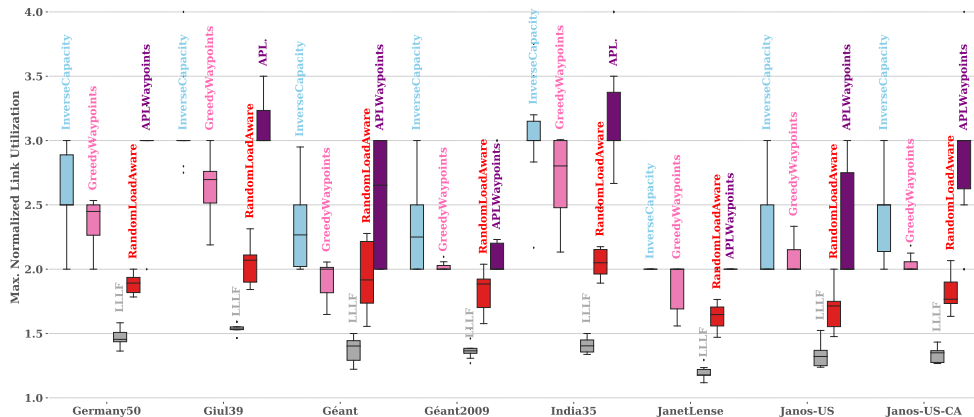


Abbildung: MLU-Value for each topology(MCF Synthetic Demands)

- **Laufzeit:** Algorithmus hat im Vergleich eine relativ geringe Laufzeit
- **Implementierung:** Unkomplizierte und intuitiv
- **Gewichtung:** Gewichtung ist anpassbar anhand von λ

- **Kompromiss:** Ergebnis ist meistens nur ein Mittelweg
- **λ -Wahl:** Schwierig gutes λ zu Wählen und große Einflüsse bei kleinen Änderungen
- **Mittelwert:** Kritische Einzelfälle oder Engpässe können übersehen werden
- **Linearkombination:** APL und MLU können nicht linear miteinander interagieren

- **Dynamische Anpassung:** Setzen λ abhängig von der Traffic Situation
- **Mehrdimensionale Zielfunktion:** Anstatt einfacher linearkombination
- **Multihop-Waypoints:** Erlauben von mehreren Zwischenknoten
- **Anpassung an Demands:** Anpassung an Latenz- und Durchsatzanforderungen der einzelnen Demands

Fragen?

Noch Fragen?

Projekt 1

Simon
Buschmann,
Yannik Buchner -
LLLF(MLU)
Nikita Podibko,
Jan Draeger - ALU
Johannes Heinrich,
Malek Haoues
Rhaïem - APL

Least Loaded Link
First (MLU)

Random Load
Aware

Average Path
Length